UNIT 3 - Question 1

Q) Distinguish between overloading using a member function and a friend function.

Answer:

Operator overloading in C++ can be done using either a member function or a friend function.

1. **Definition**
   - **Member function:** The operator function is defined inside the class.
   - **Friend function:** The operator function is declared with friend inside the class and defined outside the class.

2. **Left-hand operand**
   - **Member function:** The left operand must be an object of the class.
   - **Friend function:** The left operand need not be a class object; both operands are passed as arguments.

3. **Arguments**
   - For unary operators:
     - Member function → no explicit argument
     - Friend function → one argument
   - For binary operators:
     - Member function → one explicit argument
     - Friend function → two arguments

4. **Access**
   - **Member function:** Can access class members directly.
   - **Friend function:** Can also access private and protected members because it is a friend of the class.

5. **Use**
   - **Member function:** Best when the left operand is the class object.
   - **Friend function:** Useful when symmetry is needed or when the left operand is not a class object.

Hence, both methods are used for operator overloading, but friend functions give more flexibility, while member functions are simpler when the operator naturally belongs to the class object.

Example:

Member function:

```
class Complex {
    int real, imag;
public:
    Complex(int r=0, int i=0){ real=r; imag=i; }
    Complex operator+(const Complex& c){
        return Complex(real + c.real, imag + c.imag);
    }
};
```

Friend function:

```
class Complex {
    int real, imag;
public:
    Complex(int r=0, int i=0){ real=r; imag=i; }
    friend Complex operator+(const Complex& a,
                             const Complex& b);
};
Complex operator+(const Complex& a, const Complex& b){
    return Complex(a.real + b.real, a.imag + b.imag);
}
```

UNIT 3 - Question 3

Q) Explain operator overloading using friend functions with syntax and an example.

Answer:

Operator overloading in C++ allows us to give special meaning to an existing operator for user-defined data types. A friend function can also be used to overload operators. A friend function is declared inside the class using the keyword friend, but it is defined outside the class. It is not a member of the class, yet it can access private and protected members of the class.

1. Syntax

```
class ClassName {
    private:
        // data members
    public:
        friend return_type operator_symbol(arguments);
};
return_type operator_symbol(arguments) {
    // function body
}
```

2. Example

Program to overload '+' operator using a friend function to add two complex numbers.

```
#include <iostream>
using namespace std;
class Complex {
    int real, imag;
  public:
    Complex(int r=0, int i=0) { real = r; imag = i; }
    friend Complex operator+(const Complex &a, const Complex &b);
    void display() {
        cout << real << " + " << imag << "i";
    }
};
Complex operator+(const Complex &a, const Complex &b) {
    return Complex(a.real + b.real, a.imag + b.imag);
}
int main() {
    Complex c1(2,3), c2(4,5), c3;
    c3 = c1 + c2;
    cout << "Sum = ";
    c3.display();
    return 0;
}
```

Output:

```
Sum = 6 + 8i
```

3. Conclusion

Thus, friend functions are useful in operator overloading when both operands need to be treated equally and when the function needs access to private data of the class.

UNIT 3 - Question 4

Q) Explain different types of inheritance supported by C++ with examples.

Answer:

Inheritance is the process by which one class acquires the properties and behavior of another class. The existing class is called the base class and the new class is called the derived class. C++ supports the following types of inheritance.

1) Single Inheritance

In single inheritance, one derived class inherits from one base class.

Example:

```
class A {
public:
    void show() { cout << "Base class"; }
};
class B : public A {
};
```

2) Multiple Inheritance

In multiple inheritance, one derived class inherits from more than one base class.

Example:

```
class A { };
class B { };
class C : public A, public B {
};
```

3) Multilevel Inheritance

In multilevel inheritance, a derived class becomes the base class for another class.

Example:

```
class A { };
class B : public A { };
class C : public B { };
```

4) Hierarchical Inheritance

In hierarchical inheritance, more than one derived class inherit from the same base class.

Example:

```
class A { };
class B : public A { };
class C : public A { };
```

5) Hybrid Inheritance

In hybrid inheritance, it is a combination of two or more types of inheritance.

Example:

```
class A { };
class B : public A { };
class C : public A { };
class D : public B, public C { };
```

Summary:

Single → one base, one derived
Multiple → many bases, one derived
Multilevel → inheritance in levels
Hierarchical → one base, many derived
Hybrid → combination of inheritance types

Thus, inheritance promotes code reusability and helps in building relationships among classes in C++.

UNIT 3 - Question 5

Q) Explain the use of Virtual Functions with an example program.

Answer:

Virtual Function:

A virtual function is a member function in a base class that is declared using the keyword virtual. It is redefined in the derived class. When a base class pointer points to a derived class object, the virtual function helps call the derived class version at run time. This is called runtime polymorphism or late binding.

Use of Virtual Functions:

1. To achieve runtime polymorphism.
2. To allow function overriding in derived classes.
3. To decide at run time which function should be executed.
4. Useful when the same interface is used for different types of objects.

Example:

Program to demonstrate virtual function in C++

```
#include <iostream>
using namespace std;
class Base {
    public:
        virtual void show() {
            cout << "Display from Base class" << endl;
        }
    };

class Derived : public Base {
    public:
        void show() override {
            cout << "Display from Derived class" << endl;
        }
    };
int main() {
    Base *ptr;
    Derived d;
    ptr = &d;
    ptr-> show();
    return 0;
}
```

Output:

```
Display from Derived class
```

Thus, virtual functions are used to implement runtime polymorphism in C++. They ensure that the correct function is called based on the object type, not the pointer type.

UNIT 3 - Question 6

Q) What is an abstract class? Demonstrate with an example.

Answer:

An abstract class in C++ is a class that contains at least one pure virtual function. A pure virtual function is declared using = 0. Objects of an abstract class cannot be created. Abstract classes are used as base classes and derived classes must implement the pure virtual function.

Syntax:

```
class Base {
public:
    virtual void show() = 0;
};
```

Example:

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void draw() = 0;
};

class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing Circle" << endl;
    }
};

int main() {
    Circle c;
    c.draw();
    return 0;
}
```

Output:

```
Drawing Circle
```

Thus, an abstract class provides a common interface for derived classes and supports runtime polymorphism in C++.

UNIT 4 - Question 1

Q) What is generic programming? Explain how templates support generic programming in C++.

Generic Programming:

Generic programming is a method of writing programs in a general form so that the same code can work with different data types. Instead of writing separate functions or classes for int, float, char, etc., we write a single template and the compiler creates the required version automatically.

How templates support generic programming in C++:

1. Templates allow us to write one function or one class that can work with many data types.
2. They reduce code duplication because the same logic need not be written again for each type.
3. They improve reusability and make programs easier to maintain.
4. The compiler generates the correct code depending on the type used.
5. Templates provide type safety, unlike macros.

Syntax of function template:

```
template <class T>
T add(T a, T b)
{
    return a + b;
}
```

Example:

Program to add two values using function template

```
#include <iostream>
using namespace std;

template <class T>
T add(T a, T b)
{
    return a + b;
}

int main()
{
    cout << add(10, 20) << endl;
    cout << add(2.5, 3.5) << endl;
    return 0;
}
```

Output:

```
30
6
```

Thus, templates are the main feature that support generic programming in C++. They help write flexible, reusable, and type-safe code.